

fuClaw AIoT

技術參考手冊

Technical Reference for System Engineers & Developers

適用平台：Realtek AmebaPro2 (AMB82-mini / HUB 8735 Ultra) / ESP32-S3-WROOM-CAM

適用傳輸：Telegram Bot / MQTT (雙平台 × 雙傳輸，共 4 種韌體變體)

Build：2026-08-09 (第四版，新增多 LLM 供應商抽象層 [Gemini / OpenAI / Grok] 、ESP32-S3-CAM

OLED 顯示模組與 /oled 工具、遠端語音聊天頁面) 作者：傅仲儀 (ChungYi Fu) | 高雄，台灣

<https://github.com/fustyles/fuClaw>

第一章 系統架構總覽

1.1 執行環境

fuClaw 是一套跨硬體廠牌的嵌入式多模態 AI 代理人框架，目前已驗證可運行於兩個獨立的 SoC 平台：Realtek AmebaPro2（RTL8735B）與 ESP32-S3-WROOM-CAM。兩者共用完全相同的 LLM 推理引擎（可於 env.json 以 llm_type 選擇 Gemini／OpenAI／Grok，詳見 2.6 節）、工具派發器、JSON 工具合約與四個 Web 前端頁面，差異僅侷限於相機擷取、儲存裝置、伺服馬達驅動、TLS Client 與 Watchdog 這五個硬體抽象點（詳見 1.5 節）。

項目	Realtek AmebaPro2 (RTL8735B)	ESP32-S3-WROOM-CAM
RAM	128 MB DDR2 (SoC 內建)	8 MB PSRAM (依模組型號)
Flash	16 MB SPI NOR (開發板外接)	16 MB (N16R8 型號)
作業系統	FreeRTOS (多工並行任務排程)	FreeRTOS (ESP-IDF 雙核心排程)
主要開發板	AMB82-mini / HUB 8735 Ultra	ESP32-S3-WROOM-CAM (ESP32-S3-WROOM-1-N16R8)

i 記憶體充裕，勿套用傳統 MCU 限制

兩個平台的 RAM／Flash 規格皆遠高於傳統 8 位元 MCU，開發與擴充時不應假設記憶體極度受限；實務瓶頸主要來自 String 頻繁配置造成的 Heap 碎片化，而非容量本身（見 7.6 節）。

1.2 系統軟體堆疊

兩個平台各自使用對應的函式庫組合達成相同的功能目標；工具的 JSON 請求／回應合約在兩者之間完全一致。

AmebaPro2 建置

函式庫 / 元件	用途	備註
WiFi.h + WiFiSSLClient	WiFi 連線與 TLS/SSL 通訊	用於 LLM API (Gemini / OpenAI / Grok) 、Telegram HTTPS 與 MQTT (TLS)
PubSubClient	MQTT Broker 訊息收發	支援 QoS 0，非阻塞 TCP 模式
ArduinoJson	JSON 序列化與反序列化	tool_call 解析核心，8 KB 文件緩衝
FreeRTOS	多工任務排程	各任務獨立堆疊，詳見 1.3 節
VideoStream	相機畫格擷取	JPEG，320×240，透過 RTSP 串流
Base64	二進位→ASCII 編碼	圖片與語音嵌入 JSON 傳輸
AmebaFatFS	SD 卡檔案讀寫	持久化設定檔與對話記憶
AmebaServo	SG90 伺服馬達控制	PWM 角度控制，0–180°

函式庫 / 元件	用途	備註
DHT	DHT11 溫濕度感測	單次觸發讀值，需間隔 ≥ 1 s

ESP32-S3-CAM 建置

函式庫 / 元件	用途	備註
WiFi.h + WiFiClientSecure	WiFi 連線與 TLS/SSL 通訊	取代 AmebaPro2 的 WiFiSSLClient
PubSubClient	MQTT Broker 訊息收發	與 AmebaPro2 版本共用相同介面
ArduinoJson	JSON 序列化與反序列化	工具合約與 AmebaPro2 完全一致
FreeRTOS	多工任務排程	ESP-IDF 雙核心排程器
esp_camera	相機畫格擷取	esp_camera_fb_get() / fb_return()，取代 VideoStream
esp_task_wdt	Watchdog 餵食	長阻塞呼叫前後明確餵食，避免看門狗重啟
SD_MMC	SD 卡檔案讀寫	取代 AmebaFatFS
ESP32Servo	SG90 伺服馬達控制	取代 AmebaServo，角度限制邏輯相同
DHT (Adafruit)	DHT11 溫濕度感測	isnan() 錯誤處理邏輯相同
Base64	二進位→ASCII 編碼	與 AmebaPro2 版本共用
U8g2	SSD1306 OLED 顯示 (I2C)	drawUTF8() 支援中文字型；僅 OLED 版變體使用，見 4.3 節 /oled

1.3 FreeRTOS 任務架構

系統啟動後，`setup()` 依所選韌體變體（Telegram Bot 或 MQTT）建立對應的並行 FreeRTOS Task。兩種傳輸變體共用 Web 與排程類任務，僅通訊收發任務不同：

Task 名稱	堆疊大小	適用變體	功能說明
task_getTelegramMessage	16 KB	Telegram Bot 版本	持續對 getUpdates API 進行 HTTPS 長輪詢，接收使用者指令與語音訊息
task_getMqttMessage	32 KB	MQTT 版本	MQTT 訊息收發、Broker 重連、Callback 處理、JPEG 影像發布
task_getRequest	16 KB	共用	HTTP Web Chat / 設定管理請求接收 (Port 81)
task_getRequestStream	16 KB	共用	HTTP 視訊串流服務 (Port 82)
task_time_scheduling	6 KB	共用，預設啟用	每 60 秒掃描 schedule.json，觸發到期排程

Task 名稱	堆疊大小	適用變體	功能說明
task_heartbeat_incompleteTaskNotifier (選用)	6 KB	共用，未建立。可於 setup() 中取消 xTaskCreate() 註解啟用。	任務，並具備 RTC 自我修復機制（見 3.6 節） 週期性未完成任務檢查（每 24 小時），掃描 historicalMessages 中 1–2 天前的非排程未完成任務並回報
task_theft_detection (選用)	6 KB	共用，未建立。可於 setup() 中取消 xTaskCreate() 註解啟用。	週期性防盜偵測（每 5 分鐘）

task_heartbeat_incompleteTaskNotifier 的機制：每 24 小時觸發一次（長時間睡眠切成 1 秒切片，避免看門狗逾時），以 <HEARTBEAT> 前綴的 workId 呼叫 llmChatRequest()，要求 LLM 掃描 historicalMessages、找出時間戳記介於 1–2 天前、尚未回報完成的「非排程」任務並整理成單一狀態報告回傳給使用者；已由獨立排程系統（task_time_scheduling）管理的任務會被明確排除，避免重複回報。

❑ 共用網路資源競爭

多個 Task 共用同一個 TLS Client 實例（AmebaPro2：WiFiSSLClient；ESP32-S3-CAM：WiFiClientSecure）。若同時發起 HTTPS 請求，將造成封包衝突導致解析失敗。

Telegram 版本設計原則：任一背景任務執行前先呼叫 botClient.stop() 並等待 2 秒，避免與長輪詢連線衝突。

MQTT 版本以 wifiClient.setNonBlockingMode() 確保 RTOS 排程器在 Broker I/O 期間不會被卡住。

ESP32-S3-CAM 版本另需注意 esp_task_wdt：長時間阻塞的 SSL 傳輸、相機擷取與 JPEG 編碼之間需穿插明確的 yield，以避免看門狗逾時重啟。

1.4 系統提示詞組建邏輯（setup() 階段）

setup() 讀取 SD 卡設定檔後，組建三個不同範圍的 LLM 系統提示詞字串，供不同情境使用（此邏輯與所選 llm_type 無關，Gemini／OpenAI／Grok 共用同一組提示詞組建流程）：

變數名稱	組成內容	使用情境
systemContent	soul.md	純自然語言對話，不需工具路由（tools = -1）
systemContentNoTools	soul.md + device.md + devicesRule	需要裝置情境但無工具呼叫的推理，例如 RTC 時區換算（tools = 0）
systemContentTools	soul.md + device.md + devicesRule + skill.md + toolsDefinition	完整工具路由，主要執行路徑（tools = 1）

llmChatRequest() 與 llmSearchRequest() 的 tools 參數在呼叫時直接選擇上述三者之一；這兩個函式僅為派工

外殼，依 `llmType` 內部呼叫對應供應商的實作（`geminiChatRequest()`／`openaiChatRequest()`／`grokChatRequest()`，詳見 2.6 節）。語音轉文字管線 `sendFileToLlm()` 則完全繞過三組系統提示詞，同樣依 `llmType` 派工至 `sendFileToGemini()`／`sendFileToOpenAI()`／`sendFileToGrok()`，僅送出音訊資料與最小化的轉錄指令，確保 `Token` 用量最小且不受工具路由邏輯干擾。

i 重要：soul.md 或 device.md 缺失會觸發自動重啟

若 `soul.md` 或 `device.md` 任一為空，`setup()` 呼叫 `NVIC_SystemReset()`（`AmebaPro2`）或對應的重啟 API（`ESP32-S3-CAM`）重啟裝置並停止初始化，防止 AI 在無系統人格和無裝置定義的狀態下運行。

1.5 跨平台可移植性：AmebaPro2 與 ESP32-S3-CAM

`fuClaw` 的 `Prompt` 驅動核心從設計之初即以跨傳輸層、跨硬體為目標。除原本的 `Realtek AmebaPro2` 系列開發板外，程式碼庫現已提供第二個完整可運作的硬體移植版本：`ESP32-S3-WROOM-CAM`。

◆ 完全不變的部分

推理與協調層在兩平台間完全相同：`llmChatRequest()`、`llmSearchRequest()`、`llmVisionRequest()`（依 `llm_type` 派工至 `Gemini`／`OpenAI`／`Grok` 各自實作）、`handleAgentResponse()`、`executeTool()`、`evaluateWorkflowContinuation()`，以及 `device.md`／`skill.md`／`soul.md` 的解析邏輯與排程引擎。

為其中一個平台撰寫的 `device.md` 或 `skill.md`，只要腳位編號符合目標板已確認的 `GPIO` 集合，即可不經修改直接在另一平台使用。

項目	AmebaPro2	ESP32-S3-CAM
相機擷取	VideoStream	esp_camera (esp_camera_fb_get / fb_return)
儲存裝置	AmebaFatFS (SD 卡)	SD_MMC
伺服驅動	AmebaServo	ESP32Servo
TLS Client	WiFiSSLClient	WiFiClientSecure
Watchdog	僅靠 RTOS 任務延遲	esp_task_wdt，於長阻塞呼叫前後明確餵食

這些差異全部侷限於各工具處理函式內的硬體抽象呼叫，每個工具回傳的 `JSON` 請求／回應合約在兩平台間完全一致，因此 `evaluateWorkflowContinuation()` 與四個前端 `Web` 頁面都不需要任何平台專屬分支。若要移植至第三種開發板，開發者只需替換上述五個硬體抽象點；推理引擎、`prompt schema`、持久化設計與所有 `Web UI` 皆可重複使用而無需修改。

第二章 通訊傳輸層與訊息處理

2.1 雙通訊模式：Telegram Bot 與 MQTT

fuClaw 提供兩種通訊變體，分別針對不同部署情境最佳化；兩者共享完全相同的 LLM 推理引擎、工具派發器與持久化記憶設計，通訊傳輸是唯一的架構差異。

Telegram Bot 版本

使用 HTTPS 長輪詢對 `getUpdates` API 進行查詢，維持一條持久的 SSL 連線。設計重點：

- `chatId` 作為天然的存取控制層：僅有設定的使用者能下達指令，無需額外驗證層。
- `telegrambotKeyboard` 在 `/help` 回應中注入持久化的回覆鍵盤，讓行動裝置可一鍵存取常用指令。
- 每次輪詢回應中擷取的 `Date:` 標頭以零額外成本提供 RTC 初始化所需的 GMT 時間。
- 語音訊息（OGG/Opus）會被下載、Base64 編碼後直接送至 `sendFileToLlm()`，再依 `llm_type` 轉發至 Gemini/OpenAI/Grok 對應的 STT 端點，整條語音轉動作管線無需任何外部儲存服務。
- 相機影格以 `multipart JPEG` 方式直接上傳至 Telegram 的 `sendPhoto` API，提供原生的聊天內照片訊息。
- `replyUserMessage()` 透過 `workId` 前綴（<BOT>、<PAGE>、<MQTT>、<TIME_SCHEDULING>、<HEARTBEAT>、<THEFT_DETECTION>）將回覆路由至正確的輸出通道，無需在整个呼叫堆疊中傳遞通道參考；只有 <PAGE> 會被導向 Web Chat（`mainPageHTML`），其餘標籤在兩種通訊變體中皆導向各自的訊息通道（Telegram 或 MQTT publish 主題）。

MQTT 版本

使用 `PubSubClient` 連接至 Broker，並使用三個專屬主題：

主題	方向	用途
xxx/subscribe	接收	接收任意 MQTT client 的使用者指令
xxx/publish	發送	發送文字回覆
xxx/publishimage	發送	發送拍攝的 JPEG 影格

- 以平台前綴加上隨機十六進位後綴產生的 `client ID`，避免多裝置共用同一 Broker 時的連線衝突。
- `wifiClient.setNonBlockingMode()` 確保 RTOS 排程器在 Broker I/O 期間不會被卡住。
- `reconnect()` 以 5 秒重試間隔進行迴圈重連，並在每次成功重連後自動重新訂閱指令主題。
- 將 JPEG 資料發布至獨立的 `publishimage` 主題，讓二進位影像負載與文字回覆流量乾淨分離，便於 Broker 端篩選。
- 標準 MQTT 協定使韌體可與任何 Broker（Mosquitto、HiveMQ、雲端 Broker）搭配使用而無需修改程式碼，只需更新 `env.json`。

◆ 架構共通性

儘管傳輸層不同，兩個版本對 `llmChatRequest()`、`llmSearchRequest()`、`llmVisionRequest()`、`handleAgentResponse()`、`executeTool()`、`evaluateWorkflowContinuation()`、所有工具處理函式與持久化層的實作完全相同。

通訊傳輸是唯一的架構差異，使兩個版本易於同步維護——這種共通性同時延伸至兩個受支援的硬體平台之間，形成「2 平台 × 2 傳輸」共 4 種韌體變體（見附錄 D）。

2.2 訊息接收路徑

fuClaw 支援兩種訊息來源，統一進入同一個處理邏輯鏈：

來源	協定	進入點函式	傳輸特性
Telegram	HTTPS 長輪詢	task_getTelegramMessage()	持續輪詢，chatId 存取控制
MQTT Broker	MQTT over TCP (或 TLS)	callback()	非同步 Callback，QoS 0
Web Chat	HTTP GET (Port 81)	task_getRequest()	同步查詢，支援純文字與 MQTT WebSocket 兩種前端

三者的訊息最終都傳入 llmChatRequest()，取得 LLM 回應後再送入 handleAgentResponse() 處理。

2.3 handleAgentResponse()：回應分流器

這是整個代理人邏輯的核心函式，負責解析 LLM 的輸出並決定下一步動作。分流規則如下：

LLM 輸出特徵	判斷結果	執行動作
以 { 開頭、以 } 結尾	單一 JSON 物件	ArduinoJson 解析 → 取出 method 與 params → executeTool()
以 [開頭、以] 結尾	JSON 陣列 (多步驟)	循序執行每個 tool_call；遇到不完整項目立即中止 (最長有效前綴)
完全等於 "NONE" (區分大小寫)	工作流程完成哨兵	不執行任何動作，結束此輪工作流程
其他純文字	自然語言回覆	輸出清理 (見下方) → replyUserMessage() 傳送
JSON 解析失敗	格式錯誤	Serial.println 記錄，不觸發任何動作

前置字元清理 (必要步驟)

handleAgentResponse() 在進行 JSON 解析前，執行以下字元替換，以處理 LLM 偶發的輸出格式汙染（無論 llm_type 選擇哪個供應商，皆套用同一套清理規則）：

```
message.replace("\\\\\"", "\\\""); // 還原跳脫引號
message.replace("\\\\\"", "\\\""); // 還原跳脫反斜線
message.replace("\\n", "");      // 移除 JSON 換行
message.replace("\\t", "");      // 移除 Tab
message.replace("\\\\-", "\\-");  // 還原 Markdown 跳脫
message.replace("\\\\*", "\\*");  // 還原 Markdown 跳脫
message.replace("\\\\_", "\\_");  // 還原 Markdown 跳脫
```

輸出清理與 Markdown 過濾 (雙路徑)

- handleAgentResponse() 在將任何自然語言回應路由給使用者之前，依輸出通道採取不同的正規化策略：
- Telegram 輸出：HTML 特殊字元（&、<、>）會被轉義，避免注入 Telegram 的 HTML 解析模式。
 - Web Chat 輸出：Markdown 格式符號（**、__、###、``、反引號、---）會被移除，* 清單標記轉換為項目符號；此清理路徑專門套用於 <PAGE> workId 的回應。

這套雙路徑清理機制確保無論所選 LLM 供應商（Gemini／OpenAI／Grok）或模型的輸出風格如何，Markdown 格式傾向都不會洩漏原始語法字元至 Telegram 聊天或 Web UI 中。

2.4 executeTool()：工具執行器

executeTool() 接收已通過 JSON 解析的 method 和 params，執行對應的硬體或系統動作。每次工具執行後，結果會寫入 historicalMessages（對話記憶）和 executeToolHistory（執行日誌）。

reCheck 旗標與工作流程評估

executeTool() 的第四個參數 reCheck 控制是否在工具執行後觸發 evaluateWorkflowContinuation()：

情境	reCheck 值	行為
JSON 陣列中的中間工具	false	執行後直接繼續，不詢問 LLM，避免浪費網路資源並不必要地膨脹對話歷史
JSON 陣列中的最後一個工具	true	執行後呼叫 evaluateWorkflowContinuation()
單一 JSON 物件	true（預設）	執行後呼叫 evaluateWorkflowContinuation()

最長有效前綴 (Longest Valid Prefix)

當 LLM 產出多步驟 JSON 陣列時，handleAgentResponse() 不採取「全有或全無」策略，而是從陣列開頭循序執行，一旦遇到不完整或格式錯誤的項目即立即中止：

```
for (int i = 0; i < toolCount; i++) {
    if (command == "" || params.isNull()) {
        Serial.println("Incomplete tool detected -> abort remaining tools");
        break;
    }
    bool isLast = (i == toolCount - 1);
    executeTool(workId, command, params, isLast);
}
```

即使 AI 產出部分錯誤的輸出，系統仍可對最大有效部分採取行動，而非整體中止。在重試成本高昂的資源受限嵌入式裝置上，這是相當務實的設計。

2.5 evaluateWorkflowContinuation()：工作流程自評

每個工具執行完畢後（當 reCheck = true），系統向 LLM 發送一個評估提示詞，詢問工作流程是否完成。評估時比對的是使用者的原始意圖（task 參數），而非僅比對上一步的執行結果，使完成度判斷更準確並降低不必要的重複動作。LLM 的可能回應如下：

- 回傳 NONE：任務完成，流程結束。
- 回傳新的 tool_call JSON：需要繼續執行，再次進入 handleAgentResponse()。
- 回傳自然語言：任務完成，傳送最終說明給使用者。

◆ NONE 哨兵的格式要求

NONE 必須完全大寫且不含任何前後空白或標點。系統使用精確字串比對（==），LLM 回傳 "None"、"none"、"完成" 等變體均不會被識別為哨兵，而是作為自然語言傳送。此規則對 Gemini／OpenAI／Grok 三種供應商一視同仁。

在 soul.md 和 skill.md 中應明確強調此格式要求。

workId：事件追蹤機制

在涉及並行多工與多模態互動的複雜環境中，系統為每個產生的工作流程或工具呼叫指派一個內嵌時間戳記的唯一識別碼——workId。此設計帶來三項核心架構優勢：

- 端對端可追溯性：workId 原生整合精確時間戳記，在韌體、前端 Web 介面與 memory.md 對話記錄之間建立統一的資料線索。
- 非同步日誌與診斷：在 Telegram 輪詢、Web 伺服器與排程器日誌可能交錯的多工環境下，workId 可作為關鍵過濾條件，簡化非同步除錯的複雜度。
- 防止重複執行的冪等性保護：利用 workId 的時間戳記特性，韌體可可靠地偵測並拒絕因網路延遲、重試機制或 Telegram 長輪詢異常造成的重複指令，確保伺服馬達旋轉、GPIO 切換等敏感硬體操作僅執行一次。

2.6 LLM API 請求類型與多供應商派工

類型	函式名稱	特性	使用情境
Chat	llmChatRequest() → geminiChatRequest()	使用 /v1beta/models/{model}:generateContent，帶完整對話歷史	主要推理路徑
Vision	llmVisionRequest() → geminiVisionRequest()	圖片以 Base64 inline_data 嵌入請求 JSON，獨立於對話歷史的無狀態呼叫	/still、/vision 工具
Search	llmSearchRequest() → geminiSearchRequest()	啟用 google_search tool，取得接地搜尋結果	/search 工具
STT（語音）	sendFileToLlm() → sendFileToGemini()	音訊以 Base64 inline_data 傳送，完全繞過三組系統提示詞，僅送出最小化轉錄指令	Telegram 語音訊息
RTC 對時	getGoogleDatetime()	直接以 HTTPS 連線 google.com:443 並擷取回應的 Date: 標頭；與 llm_type 選擇無關（不消耗 LLM API 配額），Telegram、MQTT 版本皆可使用；連線失敗時退回接地搜尋提示詞	/syncrtc、開機對時

i Vision 請求的效能開銷

一張 320×240 JPEG（約 20 KB）Base64 編碼後約 27 KB，完整請求大小可達 50–100 KB。

Base64 編碼約需 27,000 次迭代，加上 HTTPS 握手與 API 回應，單次 /vision 呼叫總耗時通常為 5–15 秒。

語音檔案（Telegram OGG/Opus 格式）下載使用 HTTP/1.0，避免分塊傳輸編碼（Chunked Transfer

Encoding) 增加解碼複雜度；Base64 緩衝區在建構請求字串後立即 free()，避免與 SSL Client 爭奪 Heap 空間。

2.7 多 LLM 供應商抽象層 (Gemini / OpenAI / Grok)

fuClaw 韌體現已將推理引擎從單一 Gemini 實作，重構為可設定的 LLM 供應商抽象層。env.json 新增 llm_type、llm_key、llm_model 三個欄位 (取代舊版的 gemini_apikey / gemini_model，詳見 3.1 節)，由 setEnvironmentSettings() 讀入 llmType / llmKey / llmModel 三個全域變數。

三個對外暴露的派工函式——llmChatRequest()、llmSearchRequest()、llmVisionRequest()——依 llmType 的值 ("gemini"、"openai" 或 "grok") 將呼叫轉發至對應供應商的實作函式；語音轉文字管線的 sendFileToLlm() 採用相同的派工模式。所有呼叫端程式碼 (handleAgentResponse()、executeTool()、evaluateWorkflowContinuation()、排程引擎與防盜偵測技能) 一律呼叫 llmXxx() 系列函式，不直接呼叫任何供應商專屬函式，因此新增供應商時無需修改呼叫端邏輯。

與供應商無關的共用變數與函式也已同步泛型化：geminiRole → llmRole (soul.md 內容)、geminiTemperature → llmTemperature、geminiMaxOutputTokens → llmMaxOutputTokens、buildGeminiMessage() → buildLlmMessage()。systemContent / systemContentNoTools / systemContentTools 三組提示詞字串的組建邏輯 (見 1.4 節) 完全不受供應商切換影響。

三個供應商目前的對照如下：

llm_type	llm_model 範例	說明
gemini	gemini-3-flash-preview	geminiChatRequest() / geminiSearchRequest() / geminiVisionRequest() / sendFileToGemini()；沿用 v1beta/models/{model}:generateContent 端點與既有 systemContent 提示詞格式
openai	gpt-5.6	openaiChatRequest() / openaiSearchRequest() / openaiVisionRequest() / sendFileToOpenAI()；改用 Chat Completions 相容格式 (messages 陣列、Authorization: Bearer llmKey)，web_search 以 tools 參數啟用
grok	grok-4.5	grokChatRequest() / grokSearchRequest() / grokVisionRequest() / sendFileToGrok()；沿用與 OpenAI 相容的請求格式，Authorization: Bearer llmKey

◆ 切換供應商免重新燒錄韌體

因三個供應商共用相同的工具路由 JSON 合約 (見 2.6、4.1–4.5 節)，更換 llm_type / llm_key / llm_model 並透過 /updateConfig 或 index.html 寫回 env.json 即可切換供應商，setup() 於下次開機時重新載入，無需修改 device.md、skill.md、soul.md 或重新燒錄韌體。

三家供應商的模型能力、延遲與 Vision / STT 支援程度不同，混合部署 (例如以 openai 作為文字聊天、以

gemini 專供 /vision) 目前尚未支援——單一裝置僅能選擇單一 llm_type。

第三章 持久化設定檔規格 (SD 卡)

所有設定檔儲存於 SD 卡根目錄。系統在 `setup()` 階段依序讀取，並在對話過程中即時更新 `memory.md`、`schedule.json` 與 `scheduleTodayExecuted.md`。

3.1 env.json — 環境憑證

JSON 格式，包含所有外部服務的連線參數。系統透過 `setEnvironmentSettings()` 解析並套用。

```
{
  "device_name": "DEVICE_NAME",
  "wifi_ssid": "YOUR_SSID",
  "wifi_pass": "YOUR_PASSWORD",
  "telegramBot_token": "xxxxxxxxxx",           // 僅 Telegram Bot 版本需要
  "telegramBot_chatID": "xxxxxxxxxx",         // 僅 Telegram Bot 版本需要
  "mqtt_server": "mqttgo.io",                 // 僅 MQTT 版本需要
  "mqtt_port": "1883",
  "mqtt_user": "",
  "mqtt_password": "",
  "mqtt_subscribeTextTopic": "fuclaw/subscribe",
  "mqtt_publishTextTopic": "fuclaw/publish",
  "mqtt_publishImageTopic": "fuclaw/publishimage",
  "llm_type": "gemini", // gemini / openai / grok
  "llm_key": "AIzaSy...",
  "llm_model": "gemini-3-flash-preview",
  "schedule_timeout": 5,
  "timezone": "Asia/Taipei"
}
```

欄位	類型	說明
device_name	String	裝置名稱，用於提示詞與日誌中的識別
wifi_ssid / wifi_pass	String	WiFi 基礎設施憑證
telegramBot_token / telegramBot_chatID	String	Telegram Bot 版本專用；chatID 同時作為存取控制層
mqtt_server / mqtt_port	String	MQTT Broker 位址與端口 (1883 = 明文 · 8883 = TLS)
mqtt_subscribeTextTopic	String	裝置訂閱的指令 Topic
mqtt_publishTextTopic	String	裝置發布文字回覆的 Topic
mqtt_publishImageTopic	String	裝置發布 JPEG 圖片的 Topic
llm_type	String	LLM 供應商選擇："gemini" / "openai" / "grok"；決定 <code>llmChatRequest()</code> 等派工函式轉發至哪一組供應商專屬實作 (見 2.7 節)
llm_key	String	所選供應商的 API Key (取代舊版 <code>gemini_apikey</code>)
llm_model	String	執行期可設定的模型識別字串 (取代舊版 <code>gemini_model</code>)，例如 <code>gemini-3-flash-preview</code> / <code>gpt-5.6</code> / <code>grok-4.5</code> ；升級或降級模型只需

欄位	類型	說明
schedule_timeout	Integer	/updateConfig 推送設定，免重新燒錄韌體 排程任務執行的容忍延遲（分鐘）；設為 0 停用容錯檢查，永遠嘗試執行任何時間已過的任務
timezone	String	IANA 時區字串，影響 RTC 對時與排程判斷
memory_backup_retention （設計提案）	Integer	時間戳記記憶備份的保留天數或檔案數上限，用於 3.5 節提出的還原機制，尚未併入韌體

✕安全注意事項

env.json 含有 API Key、WiFi 密碼與 Telegram Bot Token 等敏感憑證，SD 卡遺失即等同憑證洩露。建議課後或測試後立即撤銷 LLM API Key (llm_key) 與 Telegram Bot Token，並為測試環境使用專屬的 MQTT Topic 名稱。

3.2 soul.md — 代理人人格定義

純文字格式，內容直接覆蓋 llmRole 變數，成為 LLM 系統提示詞的第一層（此變數與所選 llm_type 無關，Gemini／OpenAI／Grok 共用同一份 soul.md）。此檔案決定 AI 的回答語氣、專業領域、決策風格，以及是否在特定情境下主動要求確認。soul.md 為空或無法讀取時，setup() 將觸發裝置重啟。

3.3 device.md — 硬體裝置映射

定義 AI 被允許直接控制的 GPIO 腳位清單，是硬體安全的第一道防線。devicesRule 規定 AI 只能操作此清單中明確列出的腳位；對於未列出的裝置，AI 必須停止並向使用者請求澄清。

device.md 現在也保留兩組預設留空的區塊，僅在實際部署需要多 Agent 協作或多通道通知時才填入內容：

- Agent 對 Agent 通訊目標：對端 fuClaw 節點的 MQTT 主題、TCP 對端端點。
- 家庭管理員通知通道：household-admin 的 Telegram chatID、Line Bot / Line Notify 端點、MQTT 通知主題。

標準格式範例（HUB 8735 Ultra 完整配置）：

```
=====
CONFIRMED HARDWARE DEVICES
=====
HUB 8735 Ultra
- Green indicator LED: pin 25
- Blue indicator LED: pin 26
- Fill light LED: pin 13
  - analog output range: 0-255
- Function button: pin 12
  - digital input only, active-low

External Modules
- Emergency button: pin 1 (digital input only, active-high)
- Light sensor module: pin 2 (analog input, range 0-1023)
- Warning light: pin 11 (PWM output, range 0-255, default 255)
```

```
- Window actuator (SG90 servo): pin 12, range 0-180
- DHT11 Temperature & Humidity Sensor: PIN 20
  - Measures: temperature (°C) and humidity (%)
  - Read mode: single trigger, returns two integers
  - Temperature range: 0-50°C, Humidity: 20-90%RH
  - Physical Rule: sensor requires ~1s between reads

# Agent-to-Agent / Notification targets (reserved, empty by default)
# peer_mqtt_topic:
# tcp_peer_endpoint:
# admin_telegram_chatID:
# admin_line_token:
```

3.4 skill.md — 自主技能定義

定義 AI 可主動執行的複合作業流程（Skill）。每個 Skill 包含目標描述、機器可讀的 JSON 步驟序列，以及條件判斷邏輯。Skill 由 periodicCheck Task 或排程觸發，執行時豁免使用者確認要求。Skill 定義必須嚴格使用 JSON 格式輸出步驟，不得包含自然語言說明。

3.5 memory.md — 對話歷史持久化

儲存 LLM API 所需的 messages 陣列格式，每次工具執行後自動附加並寫回 SD 卡。系統重啟後讀取此檔案，還原 historicalMessages 變數，實現跨重啟的記憶連續性。

□ Heap 碎片化風險 與 原子化寫入保護

隨對話增加，memory.md 持續成長，String 型態的頻繁 malloc/free 操作會導致 Heap 碎片化；建議在 historicalMessages 長度超過 50,000 字元時定期執行 /reset。

storeDataToFile() 在每次對話更新後即時呼叫，並採「先重新命名為 memory.md.bak，再寫入新版本」的兩段式策略：即使寫入過程中斷電，先前備份仍完整保留，不會損毀唯一一份對話歷史。

時間戳記備份與還原機制（設計提案）

現行 memory.md.bak 僅保留「上一版」備份，一旦連續寫入兩次，較舊的版本即被覆蓋，無法回溯到更早的對話狀態。以下為擴充為多版本時間戳記備份的設計提案，尚未併入韌體，記錄於此供後續實作參考。

- 命名規則：/reset 觸發備份時，以 RTC 時間組出 memory_YYYYMMDD_HHMMSS.bak（例如 memory_20260705_153000.bak），取代單一固定檔名的 memory.md.bak，使 SD 卡上可同時保留多個歷史版本。
- RTC 未就緒的邊界情況：產生檔名前檢查 rtcYear == 0；若 RTC 尚未完成 /syncrtc，退回以開機次數與序號組合命名（例如 memory_boot3_seq1.bak），避免時間戳記錯誤或檔名衝突。
- 保留策略：新增 env.json 欄位 memory_backup_retention（單位：天數或檔案數上限），由 Web 介面定期清理超出保留範圍的最舊備份，避免 SD 卡上的備份清單無限增長。
- 危險護欄：任何一次「還原」動作，必須先把目前使用中的 memory.md 也存一份新備份再覆蓋，避免使用者選錯日期就永久遺失當下對話——與現行 storeDataToFile() 的兩段式寫入邏輯精神一致。
- 生效方式：由於 historicalMessages 僅在開機時從 memory.md 讀入記憶體，還原檔案後應比照 /updateConfig 的既有模式，直接觸發裝置重開機以套用還原結果，而非嘗試熱重載。

✗尚待補強：管理端點目前無存取驗證

還原機制涉及的 Web 端點（見 7.2 節）會直接覆蓋對話記憶並觸發重開機，屬於會改變裝置狀態的操作。目前 port 81 上的管理端點皆無任何身分驗證，僅適用於受信任的區域網路環境；若考慮曝露於公網，應在此功能落地前一併補上基本驗證機制。

3.6 schedule.json 與排程雙模式

由 /schedule 工具或 Web 排程介面寫入，儲存使用者設定的定時任務列表。task_time_scheduling Task 每 60 秒掃描此檔案，比對當前 RTC 時間，觸發到期任務。排程任務執行時，自動豁免使用者確認要求（代表排程本身即為事先授權）。

雙模式排程管理

- AI 自然語言解析模式：使用者可透過 Telegram 或聊天介面輸入日常語言（例如「每週一到五早上 8:30 設定防盜偵測」），所設定的 LLM 自動解析意圖與時間參數，轉換為結構化 JSON 任務格式，經本地邊界安全驗證後即時寫入 schedule.json。
- 手動圖形化 Web UI 模式：內建 index_schedule.html 排程管理介面，供離線或安靜環境下以標準圖形介面手動新增、編輯、修改或刪除任何排程任務。

兩種路徑即時讀寫同一份 schedule.json 核心檔案，達成「高度彈性的自然語言輸入」與「高度確定性的圖形化管理」之間的完美協調。

RTC 對時與自我修復

- Telegram 版本：getTelegramMessage() 輪詢迴圈同時擷取 HTTP 回應標頭的 Date: 欄位作為 GMT 時間來源，零額外網路成本。
- 通用做法：getGoogleDatetime() 直接對 google.com 發起輕量 HTTPS 請求並擷取 Date: 標頭，與所選 llm_type 無關，適用於 Telegram 與 MQTT 兩種版本；連線失敗時退回接地搜尋提示詞。
- rtcInitialTime() 以角色專用系統提示詞（tools = -1）透過 llmChatRequest() 請所選 LLM 將 GMT 時間換算為設定時區，並加上 4 秒傳播延遲補償；提示詞強制要求純 JSON 回應（無 Markdown、首尾字元須為 { 與 }）。
- task_time_scheduling 每次評估前檢查 rtcYear == 0；若 RTC 尚未初始化，先嘗試呼叫 /syncrtc 自我修復，僅在修復仍失敗時才跳過本次週期，避免因暫時性 RTC 初始化失敗而錯過排程。

scheduleTodayExecuted.md 與排程狀態工具

此檔案記錄當日已執行過的排程任務，避免重複觸發的排程在同一天內再度執行。搭配 /getUnfinishedSchedule 與 /updateScheduleStatus 兩項工具（詳見 4.4 節），LLM 可主動查詢尚未完成的排程並回報執行狀態，形成完整的排程生命週期管理閉環。

3.7 Web 介面與其他持久化檔案總覽

檔案	說明
index.html	設定管理介面（含 llm_type、llm_model、schedule_timeout 顯示與變更）

檔案	說明
index_agent.html	Agent 管理介面：檢視 / 編輯 soul.md、device.md、skill.md
index_schedule.html	排程管理介面：圖形化新增、編輯、刪除排程任務
index_chat.html	LLM Web 聊天介面（純 HTTP GET /message 查詢）
index_mqtt_chat.html	透過 MQTT WebSocket 的 LLM Web 聊天介面（即時雙向串流；獨立的遠端語音版見 7.2 節）
index_restore.html（設計提案）	記憶備份還原介面：列出時間戳記備份清單，供使用者選擇還原至指定日期的對話狀態，詳見 3.5 節
scheduleTodayExecuted.md	記錄當日已執行的排程任務，防止當日重複觸發

所有設定檔完全解耦，任一檔案均可獨立修改而無需重新燒錄韌體。`env.json` 中的憑證於開機時最先載入，使同一份韌體二進位檔可部署至多台配置各異的裝置——包括每台裝置各自對應的 LLM 供應商與模型等級（`llm_type`／`llm_model`）。

第四章 工具 API 完整規格

所有工具均以 JSON `tool_call` 格式呼叫。LLM 輸出的 JSON 由 `handleAgentResponse()` 解析後，送入 `executeTool()` 執行。以下為每個工具的完整請求／回應格式；此合約在 AmebaPro2 與 ESP32-S3-CAM 兩平台之間、以及 Gemini／OpenAI／Grok 三種 `llm_type` 之間完全一致。

4.1 GPIO 輸出輸入

/digitalwrite — 數位輸出

```
// 請求
{ "type": "tool_call", "method": "/digitalwrite",
  "params": { "pin": <int>, "pinmode": "digitalwrite", "value": 0|1 } }
// 成功回應
{ "status": "success", "method": "digitalwrite", "workId": "<system>" }
// 錯誤回應 (value 不為 0 或 1)
{ "status": "error", "method": "digitalwrite",
  "reason": "invalid_digital_value", "workId": "<system>" }
```

/analogwrite — 類比輸出 (PWM)

```
// 請求
{ "type": "tool_call", "method": "/analogwrite",
  "params": { "pin": <int>, "pinmode": "analogwrite", "value": 0-255 } }
// 成功回應
{ "status": "success", "method": "analogwrite", "workId": "<system>" }
// value 由 constrain() 強制限制在 0-255，不會拋出範圍錯誤
```

/digitalread — 數位輸入

```
{ "type": "tool_call", "method": "/digitalread",
  "params": { "pin": <int>, "pinmode": "digitalread" } }
// 成功回應
{ "status": "success", "method": "digitalread", "value": 0|1, "workId": "<system>" }
```

/analogread — 類比輸入 (ADC)

```
{ "type": "tool_call", "method": "/analogread",
  "params": { "pin": <int>, "pinmode": "analogread" } }
// 成功回應 (ADC 原始值)
{ "status": "success", "method": "analogread", "value": 0-1023, "workId": "<system>" }
```

4.2 視覺與搜尋

/still — 拍照並傳送

```
{ "type": "tool_call", "method": "/still",
  "params": {
    "frames": true,    // true = 擷取新畫格；false = 使用上次快取
    "task": "NONE"    // 後續任務描述，無則填 NONE
  }
}
```

```
}
```

/vision — 拍照並由 LLM 分析

```
{ "type": "tool_call", "method": "/vision",  
  "params": {  
    "query": "分析圖片中的內容",  
    "frames": true,  
    "task": "分析完成後的後續動作，無則填 NONE"  
  }  
}
```

frames: false 的設計目的：當 /vision 已擷取並快取圖片後，後續的 /still 使用 frames: false 可傳送同一張圖片，確保警示照片與偵測時間點一致，而非重新擷取（人可能已離開）。若 frames 為 false 且緩衝區內無先前影像（imageLength == 0），系統會提早回傳錯誤而非以空緩衝區繼續執行。ESP32-S3-CAM 版本以 esp_camera 的畫格緩衝區（esp_camera_fb_get() / esp_camera_fb_return()）實作相同的快取合約，Prompt 層語意完全不變。

/search — 網路接地搜尋

```
{ "type": "tool_call", "method": "/search",  
  "params": {  
    "query": "搜尋內容",  
    "task": "搜尋結果取得後的後續動作，無則填 NONE"  
  }  
}
```

4.3 擴充感測器

/dht11 — DHT11 溫濕度感測器

```
// 請求  
{ "type": "tool_call", "method": "/dht11", "params": { "pin": <int> } }  
// AMB82-mini -> pin 8; HUB 8735 Ultra -> pin 20  
// 成功回應  
{ "status": "success", "method": "/dht11",  
  "temperature": <float>, "humidity": <float>, "workId": "<system>" }  
// 錯誤回應 (isnan() 檢測失敗)  
{ "status": "error", "method": "/dht11",  
  "reason": "dht11_read_failed", "workId": "<system>" }
```

/servo — SG90 伺服馬達控制

```
// 請求  
{ "type": "tool_call", "method": "/servo", "params": { "pin": <int>, "angle": 0-180 } }  
// AMB82-mini window actuator -> pin 5; HUB 8735 Ultra -> pin 12  
// 成功回應  
{ "status": "success", "method": "/servo", "workId": "<system>" }  
// 錯誤回應 (角度超出範圍 或 未定義的腳位)  
{ "status": "error", "method": "/servo",  
  "reason": "invalid_servo_angle | undefined_servo_pin: <pin>", "workId": "<system>" }
```

/oled — SSD1306 OLED 文字顯示 (ESP32-S3-CAM OLED 版變體專屬)

```
// 請求
{ "type": "tool_call", "method": "/oled", "params": { "line1": "<text>", "line2": "<text>",
"line3": "<text>", "line4": "<text>" } }
// 僅 ESP32S3_CAM_fuClaw*_OLED.ino 變體提供；AmebaPro2 (AMB82-mini/HUB 8735 Ultra) 未配備此硬體，無
tool_oled() 實作
// 空字串清除該行；每行以 u8g2_font_unifont_t_chinese1 字型繪製，支援中文 UTF-8
// 成功回應
{ "status": "success", "method": "/oled", "workId": "<system>" }
// 錯誤回應
{ "status": "error", "method": "/oled",
"reason": "<error reason>", "workId": "<system>" }
```

硬體：I2C 軟體匯流排，SDA=GPIO1、SCL=GPIO2 (U8G2_SSD1306_128X64_NONAME_F_SW_I2C)，128×64 解析度，四行文字各佔 16px。驅動函式 tool_oled() 於 setup() 呼叫 u8g2.begin() 初始化；executeTool() 收到 method == "/oled" 時呼叫此函式 (見 1.2 節 U8g2 函式庫列與附錄 A)。

4.4 系統管理工具

工具	說明	回應
/delay	暫停執行 0–10000 ms (constrain() 強制限制)	success + workId
/getMemory	回傳 Heap 可用空間與 historicalMessages 長度	success + 記憶體數值
/getLog	回傳 executeToolHistory 字串 (Serial 輸出)	success
/reset	清除 historicalMessages，還原初始提示詞狀態	success，對話記憶清空
/chat	純自然語言回覆，包含 reply 文字	傳送給使用者
/reboot	重啟裝置	裝置重啟
/schedule	新增排程任務至 schedule.json	success + workId
/getSchedule	取得所有排程任務	success + 任務列表
/getUnfinishedSchedule	取得尚未執行完成的排程任務清單，供 LLM 主動查詢待辦排程	success + 未完成任務列表
/updateScheduleStatus	更新指定排程任務的執行狀態 (例如標記為已完成)，寫回 schedule.json 並同步 scheduleTodayExecuted.md	success + workId
/modifySchedule	修改或刪除指定排程任務	success + workId
/clearSchedule	清除所有排程任務	success
/syncrtc	透過網路時間對時 RTC	success + 時間字串
/getrtc	取得 RTC 當前時間	success + 時間字串

Agent 對 Agent 與多通道通訊工具（`/tcpSendMessage`、`/mqttSendMessage`、`/mqttSendImage`、`/telegramSendMessage`、`/telegramSendImage`、`/lineSendMessage`）獨立於本節之外，因其涉及跨裝置協作與安全邊界考量，完整規格另見 4.5 節。

4.5 Agent 對 Agent 與多通道通訊工具

為了從單一邊緣裝置進化為協作式的多 Agent 生態系統，fuClaw 以原生自主通訊工具擴充其 Prompt 驅動工具路由機制。這些工具讓 LLM 推理引擎不僅能操作本地 GPIO，更能自主決定何時將狀態遙測、文字告警或原始二進位資料，透過 P2P 網路、MQTT Broker、Telegram 與 Line 進行傳播。目標位址存放於 `device.md` 的保留區塊中（見 3.3 節），僅在實際部署需要時才填入。

方法	職責	限制與安全邊界
<code>/tcpSendMessage</code>	直接以 P2P 文字訊息傳送給另一台指定的 fuClaw 節點	目標須網路可達；用於低開銷的邊緣對邊緣同步
<code>/mqttSendMessage</code>	透過設定的 Broker，將文字負載發布至指定 MQTT 主題	依賴 <code>env.json</code> 中的連線狀態；Broker 中斷時回傳錯誤 JSON
<code>/mqttSendImage</code>	擷取並推送目前相機快照至指定 MQTT 主題	受 FreeRTOS stack 防護監控；大型影像封包使用自動動態分塊緩衝區
<code>/telegramSendMessage</code>	直接將文字告警推送至設定的 Telegram 聊天，獨立於輪詢迴圈之外	重複使用 <code>env.json</code> 中的 bot token / chatID；不需要先有使用者訊息即可觸發
<code>/telegramSendImage</code>	將目前相機影格以原生照片訊息推送至 Telegram	與 <code>/still</code> 相同的影格快取規則；傳輸大小受可用 heap 限制
<code>/lineSendMessage</code>	將文字通知發布至設定的 Line Bot / Line Notify 端點	須提供有效的 Line 頻道 token；傳送失敗時回傳錯誤 JSON

◆一致的工具合約

上述工具皆遵循與所有既有工具相同的 JSON 回應合約——`status`（`success` 或 `error`）、識別工具的 `method`，以及結果資料或失敗時的 `reason` 欄位。這種一致性讓 `evaluateWorkflowContinuation()` 能以統一方式推理任何工具結果，無論底層硬體類型或平台為何。

架構設計重點

- 去中心化多 Agent 協作（`/tcpSendMessage`）：當 LLM 引擎評估本地感測器異常（例如 DHT11 溫度超過閾值）並判斷需要遠端實體區域共同介入時，便會呼叫 `/tcpSendMessage`，繞過集中式伺服器邏輯，讓邊緣 Agent 在應用層直接協商動作，並將操作情境保留於 `memory.md` 中。
- 非同步 IoT 發布/訂閱拓樸（`/mqttSendMessage`）：能進行次秒級的遙測廣播，訊息直接送往 `env.json` 設定的主題，與工業 IoT 平台、Home Assistant、Node-RED 或自訂的 ESP32/Ameba 子節點開箱即用整合。
- 影格重用與記憶體保護（`/mqttSendImage`、`/telegramSendImage`）：延續 `/still` 與 `/vision` 的行為規則，`frames: false` 可強制工具重用先前 Vision 任務鎖定的既有 JPEG 緩衝區；執行區塊在

`task_getMqttMessage` 緊湊的 stack（32 KB）之外動態配置記憶體，確保網路 socket 與 SSL 交握過程不會與核心 runtime 爭奪記憶體。

- 跨通道通知備援（`/telegramSendMessage`、`/lineSendMessage`）：這兩項工具可在任何推理情境下被觸發——排程任務、防盜偵測或互動聊天——Agent 可同時將同一則告警分發至 Telegram 與 Line，讓部署環境內建通知備援路徑，無需額外的外部自動化伺服器。

第五章 新感測器擴充 SOP

fuClaw 採用「四步驟擴充模型」整合新硬體，以下以 DHT11 和 SG90 為完整示範，可直接作為其他感測器的擴充模板。此模型同時適用於 AmebaPro2 與 ESP32-S3-CAM 兩平台，差異僅在步驟三的底層驅動函式庫（見 1.5 節對照表）。

5.1 擴充四步驟

步驟	操作	修改位置
步驟 1	在 device.md 加入硬體描述與物理規則	SD 卡（不需重新燒錄）
步驟 2	在 toolsDefinition 字串中加入工具 JSON 規格	韌體原始碼（需重新燒錄）
步驟 3	撰寫底層驅動函式 tool_xxx()	韌體原始碼（需重新燒錄）
步驟 4	在 executeTool() 中加入 else if 分支	韌體原始碼（需重新燒錄）

5.2 步驟一：device.md 擴充

在 device.md 的「External Modules」段落加入感測器描述。物理規則（範圍、讀取模式、時序要求）必須明確標注，LLM 才能正確決策何時可以呼叫工具：

```
- DHT11 Temperature & Humidity Sensor
- Pin mapping:
  AMB82-mini: PIN 8
  HUB 8735 Ultra: PIN 20
- Measures: temperature (°C) and relative humidity (%)
- Read mode: single trigger, returns two integer values
- Temperature range: 0-50°C
- Humidity range: 20-90% RH
- Physical Rule: Values are integers. Sensor requires ~1s between reads.
```

5.3 步驟二：toolsDefinition 工具規格

在韌體的 toolsDefinition 字串中加入新工具的 JSON 請求／回應格式。此定義送給 LLM 作為提示詞，告知 AI 何時及如何呼叫此工具：

```
Reading the DHT11 temperature and humidity sensor:
{
  "type": "tool_call",
  "method": "/dht11",
  "params": {
    "pin": "<Device pin number. If the user does not specify a pin, ask first.>"
  }
}
Success response:
{ "status": "success", "method": "/dht11",
  "temperature": <value>, "humidity": <value>, "workId": "<system-provided>" }
Error response:
{ "status": "error", "method": "/dht11",
  "reason": "<error reason>", "workId": "<system-provided>" }
```


5.4 步驟三：驅動函式

撰寫底層驅動，處理硬體讀取與 JSON 回應組建。以 DHT11 為例（AmebaPro2 / ESP32-S3-CAM 共用相同的錯誤處理邏輯，僅函式庫標頭不同）：

```
#include "DHT.h"    // ESP32-S3-CAM 使用 Adafruit DHT 函式庫
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE); // DHTPIN 依開發板設定

String tool_dht11(int pin, String workId) {
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (isnan(h) || isnan(t)) {
        return
        "{\"status\":\"error\",\"method\":\"/dht11\",\"reason\":\"dht11_read_failed\",\"workId\":\"" +
        workId + "\"}";
    }
    return "{\"status\":\"success\",\"method\":\"/dht11\",\"temperature\":\"" + String(t) +
    "\",\"humidity\":\"" + String(h) + "\",\"workId\":\"" + workId + "\"}";
}
// setup() 中加入初始化：
delay(1000); // 等待感測器穩定
dht.begin();
```

SG90 伺服馬達驅動函式以相同模式撰寫，額外需要驗證角度範圍（0–180）並在超出範圍時回傳 `invalid_servo_angle` 錯誤（AmebaPro2 使用 `AmebaServo`，ESP32-S3-CAM 使用 `ESP32Servo`，API 介面相容）：

```
#include <AmebaServo.h>    // ESP32-S3-CAM: #include <ESP32Servo.h>
AmebaServo servol2;

String tool_servo(AmebaServo &servo, int pin, int angle, String workId) {
    if (!servo.attached()) servo.attach(pin);
    if (angle < 0 || angle > 180) {
        return
        "{\"status\":\"error\",\"method\":\"/servo\",\"reason\":\"invalid_servo_angle\",\"workId\":\"" +
        workId + "\"}";
    }
    servo.write(angle);
    return "{\"status\":\"success\",\"method\":\"/servo\",\"workId\":\"" + workId + "\"}";
}
// setup() 中：servol2.attach(12);
```

5.5 步驟四：executeTool() 分支

在 `executeTool()` 函式中，於現有的 `else if` 鏈中加入新工具的分支。以下為 DHT11 和 SG90 的完整分支範例：

```
else if (command == "/dht11") {
    int pin = params["pin"].as<int>();
    String response = tool_dht11(pin, workId);
    historicalMessages += buildLlmMessage("user", command + timestamps);
    historicalMessages += buildLlmMessage("model", response + timestamps);
    executeToolHistory += workId + " " + command + " [ " + String(pin) + " | " + response +
    " ]\n";
    evaluateWorkflowContinuation(workId, reCheck);
}
else if (command == "/servo") {
    int pin = params["pin"].as<int>();
    int angle = params["angle"].as<int>();
    String response = "";
    if (pin == 12) response = tool_servo(servol2, pin, angle, workId);
    else response = "{\"status\":\"error\",\"reason\":\"undefined_servo_pin: " + String(pin) +
```

```
"\", \"workId\": \"\" + workId + \"\"}";
historicalMessages += buildLlmMessage("user", command + timestamps);
historicalMessages += buildLlmMessage("model", response + timestamps);
executeToolHistory += workId + " " + command + " [ " + String(pin) + " | " + String(angle) +
" ]\n";
evaluateWorkflowContinuation(workId, reCheck);
}
```

第六章 硬體安全規範

6.1 裝置控制政策 (Global Device Control Policy)

所有直接由使用者請求觸發的硬體輸出動作，執行前必須獲得明確確認。以下為三層優先順序：

優先級	規則	說明
1 (最高)	安全限制	未確認的腳位絕不操作，未定義的裝置絕不猜測
2	排程任務執行規則	排程任務代表事先授權，執行時免除確認要求
3	自主工作流程規則	Skill 觸發的動作 (如防盜偵測) 免除確認要求
4 (最低)	一般確認要求	使用者直接請求的硬體控制須先確認

6.2 原子執行規則 (Atomic Execution Rule)

每次工具呼叫必須是最小不可分割的單元：

- 一個腳位 (One Pin)：不能在同一個 tool_call 中同時操作多個 GPIO。
- 一個操作 (One Operation)：不能同時讀取和寫入。
- 一個數值 (One Value)：digitalwrite 只能是整數 0 或 1，不能是字串或複合指令。

多步驟操作必須使用 JSON 陣列，由系統循序執行（見 2.4 節最長有效前綴）。若允許單一指令同時操作多個腳位，執行中途失敗將使系統陷入無法判定的半完成狀態，可能造成裝置損壞或安全隱患；原子性確保每一步驟皆完整且可驗證。

6.3 數值驗證規則

工具	有效範圍	越界行為
/digitalwrite	0 或 1 (整數)	回傳 invalid_digital_value，不執行
/analogwrite	0–255 (整數)	constrain() 強制限制，不報錯
/delay	0–10000 (毫秒)	constrain() 強制限制，不報錯
/servo	0–180 (角度整數)	回傳 invalid_servo_angle，不執行
/digitalread	無需輸入數值	純讀取，回傳 0 或 1
/analogread	無需輸入數值	純讀取，回傳 ADC 原始值 0–1023

6.4 輸出裝置狀態查詢規則

當使用者詢問輸出裝置（LED、繼電器等）的當前狀態時，AI 必須從對話歷史和工具執行歷史判斷，不得呼叫 /digitalread 或 /analogread 來讀取輸出腳位的電氣狀態。

i 為何不用 `digitalread` 查輸出狀態？

裝置邏輯狀態（on/off）與 GPIO 電氣電位（0/1）是不同概念。直接讀取輸出腳位的電位可能因硬體差異或外部干擾而產生誤判，且無法反映 PWM 亮度等類比狀態。從對話歷史判斷是更可靠的設計。

第七章 Web 管理介面與部署維運

7.1 首次部署步驟

- 選擇韌體變體：依硬體平台（AmebaPro2 / ESP32-S3-CAM）與通訊傳輸（Telegram Bot / MQTT）挑選對應的 .ino 檔案，共 4 種組合（見附錄 D）。
- 格式化 SD 卡（FAT32），建立設定檔：env.json、soul.md、device.md、skill.md、memory.md（schedule.json 與 scheduleTodayExecuted.md 會於首次排程操作時自動建立）。
- 填寫 env.json：WiFi 憑證、Telegram Bot Token/ChatID 或 MQTT Broker 參數（依所選變體）、LLM 供應商選擇與憑證（llm_type／llm_key／llm_model，見 3.1、2.7 節）。
- 依實際硬體接線更新 device.md。
- 在 Arduino IDE 安裝對應 SDK（Realtek AmebaPro2 SDK 或 ESP32 開發板套件）與必要函式庫（DHT、AmebaServo/ESP32Servo、ArduinoJson、PubSubClient）。
- 燒錄韌體至開發板，插入 SD 卡，接通電源。
- 觀察 Serial Monitor（115200 baud）確認啟動序列：env.json 讀取 → WiFi 連線 → 各設定檔載入 → FreeRTOS Task 建立。
- LED 閃爍三次（WiFi 連線成功），Telegram 輪詢或 MQTT 訂閱完成後系統進入就緒狀態。

7.2 Web 管理介面總覽

系統同時啟動兩個 HTTP 服務：port 81 提供輕量管理伺服器（task_getRequest），port 82 直接從相機串流即時 MJPEG 影像。

端點	功能
GET /	提供已預填目前憑證的 index.html
GET /updateConfig?{json}	將 env.json 儲存至儲存裝置並觸發自動重開機
GET /agent	提供 index_agent.html (Agent 管理介面)
GET /getSoul	回傳 Soul 內容
GET /updateSoul?{data}	以新內容覆寫 Soul 定義
GET /getDevice	回傳 Device 內容
GET /updateDevice?{data}	以新內容覆寫裝置定義
GET /getSkill	回傳 Skill 內容
GET /updateSkill?{data}	以新內容覆寫技能定義
GET /schedule	提供 index_schedule.html (排程管理介面)
GET /getScheduleTasks	回傳原始 schedule.json 內容

端點	功能
GET /updateScheduleTasks?{json}	以新任務陣列覆寫 schedule.json
GET /chat	提供 index_chat.html (LLM Web 聊天介面)
GET /mqtt	提供 index_mqtt_chat.html (透過 MQTT 的 LLM Web 聊天介面)
GET /message?{text}	處理聊天訊息並回傳 AI 回覆

/updateConfig 端點會在寫入儲存裝置前驗證傳入的內容是否為完整 JSON 物件（`startsWith("{")` && `endsWith("}")`），避免儲存到部分或損毀的設定。設定頁面也會顯示目前選用的 `llm_type`、`llm_model` 與 `schedule_timeout`，讓使用者可直接在瀏覽器中檢視與變更，無需修改原始碼。

雙 AP+STA 並行模式

裝置同時啟動一個 Access Point（192.168.1.1:81）與一個 Station 連線，即使家中 WiFi 無法使用，裝置仍可供設定連線——這對初次設定與現場復原十分關鍵。

Web 聊天介面 (index_chat.html)

聊天頁面透過 GET /message?<text> 與裝置溝通——純 HTTP 查詢，無需 WebSocket 或後端伺服器。設計重點：

- 輸入框會隨內容自動伸縮高度，維持行動裝置畫面整潔。
- 三點跳動動畫作為輸入指示，避免重複送出。
- 當回應內容含 `data:image` 時，訊息泡泡會切換為 HTML 渲染模式，直接於聊天內顯示拍攝影格。
- 網路失敗會以限時浮層提示，不破壞 UI 狀態。
- `handleAgentResponse()` 針對 <PAGE> `workId` 採用獨立的清理路徑，將 * 清單標記轉為 • 項目符號並移除程式碼區塊標記，產出乾淨可讀、不含原始 Markdown 語法的內容。

透過 MQTT 的 Web 聊天介面 (index_mqtt_chat.html)

此介面為需要持續雙向串流的情境而設計，直接透過 WebSocket 連接至 MQTT Broker，在瀏覽器與 fuClaw 之間建立即時發布/訂閱通道，無需輪詢。重點功能：

- 側邊欄可即時輸入 Broker 位址、Port、帳號與密碼，無需修改任何程式碼，並以可摺疊區塊保持介面簡潔。
- 頂端的彩色圓點徽章一目了然地呈現連線狀態：常亮綠燈（已連線）、閃爍琥珀（連線中）、紅色（已斷線）。
- 主題可在 client 運作期間動態訂閱或取消訂閱，每個主題可獨立指定訊息格式（TEXT / HTML / BASE64 / BIN），連線後立即生效，主題清單以色塊標示格式類型。
- 收到的內容會依各主題格式自動處理——純文字原樣顯示、HTML 原生渲染、Base64 字串與二進位 Buffer 皆會解碼為內嵌影像，讓相機影格能直接顯示於聊天泡泡中。
- 每則訊息會標示其來源主題名稱，讓多主題對話保持清楚有條理。
- 主題比對邏輯完整實作 +（單層）與 #（多層）萬用字元，完全符合 MQTT 規範。

遠端語音聊天頁面 (fuClaw_mqtt_chat.html)

index_mqtt_chat.html 由裝置自身的 HTTP 伺服器提供（GET /mqtt，見 7.2 節），但瀏覽器的麥克風擷取 API（getUserMedia/SpeechRecognition）僅在安全情境（Secure Context）下可用——https:// 或 localhost。裝置的 Web 伺服器僅提供明文 HTTP，因此裝置端頁面本身不含語音輸入功能。

為此，專案另外提供 fuClaw_mqtt_chat.html，一份以 https 方式託管於 GitHub Pages（https://fustyles.github.io/webduino/fuClaw_mqtt_chat.html）的獨立版本，與 index_mqtt_chat.html 共用相同的介面與 MQTT WebSocket 邏輯，額外新增：

- 麥克風輸入按鈕（Web Speech API，SpeechRecognition），可直接以語音輸入指令，辨識完成後填入訊息框。
- 連線設定新增「Use TLS (wss://)」核取方塊；頁面偵測到自身以 https:// 載入時會自動勾選，改用 wss:// 連往 Broker，避免瀏覽器封鎖來自 https 頁面的明文 ws:// 連線（Mixed Content）。

index_mqtt_chat.html 的工具列右上角提供「Voice Chat (Remote)」連結，直接開啟此遠端語音版頁面；兩者操作裝置的方式完全相同，僅語音輸入與連線安全性有別。

i 憑證仍在瀏覽器本機處理

遠端頁面雖由 GitHub Pages 託管，Broker 位址、帳密與 Topic 仍由使用者於瀏覽器端輸入並僅存於當前分頁的記憶體中，直接以瀏覽器對 Broker 建立 WebSocket 連線；fuClaw 裝置本身不會將任何憑證回傳至 GitHub Pages，此頁面純粹作為靜態前端資源。

7.3 Serial Monitor 啟動序列解讀

```
env.json len: 312          // env.json 讀取成功
Soul.md len: 128           // soul.md 讀取成功
device.md len: 847         // device.md 讀取成功
skill.md len: 623          // skill.md 讀取成功
memory.md len: 2048        // 對話歷史還原
AP ssid : fuclaw           // AP 模式 SSID
AP password : 12345678     // AP 模式密碼
fuClaw Manager: http://192.168.x.x:81
Video stream: http://192.168.x.x:82
RTC START: 2026-07-05 .. // RTC 對時完成
```

□ 啟動失敗常見原因

soul.md 或 device.md 長度為 0：系統會輸出 "System configuration failed" 並在 5 秒後重啟。請確認 SD 卡根目錄有這兩個檔案且內容不為空。

WiFi 連線失敗：裝置仍可運行（AP 模式），但 LLM API、Telegram 與 MQTT 皆無法使用。

Telegram 輪詢失敗（Telegram Bot 版本）：task_getTelegramMessage 會持續重試，不影響 Web Chat 功能。

MQTT 連線失敗（MQTT 版本）：task_getMqttMessage 會持續嘗試重連，不影響 Web Chat 功能。

7.4 記憶體監控與維護

指令	功能	建議閾值
----	----	------

指令	功能	建議閾值
/getMemory	查看 Heap 剩餘空間與歷史最低值	低於 50 KB 時執行 /reset
/reset	清除對話歷史，釋放 historicalMessages 佔用的 Heap	historicalMessages > 50,000 字元時
/getLog	查看工具執行歷史記錄（Serial 輸出）	除錯用途
/reboot	重啟裝置，完全釋放 Heap 碎片	Heap 嚴重碎片化時

7.5 已知技術限制

限制項目	說明	緩解策略
Heap 碎片化	String 型態的頻繁 malloc/free，長時間運行後可能導致記憶體不足	定期 /reset；監控 xPortGetMinimumEverFreeHeapSize()
網路資源競爭	多個 FreeRTOS Task 共用同一 TLS Client	架構設計確保同時只有一個 Task 發起 HTTPS 請求
LLM 輸出不確定性	Temperature=1.0 時，偶發 JSON 格式汙染或 NONE 哨兵變體	handleAgentResponse() 前置清理；在 soul.md 強調格式要求
Vision 任務高延遲	Base64 編碼 + HTTPS + API 回應，通常 5–15 秒	非即時場景使用；不適合需要毫秒級反應的安全系統
ArduinoJson 緩衝限制	DynamicJsonDocument 設為 8 KB；超大 JSON 可能解析失敗	調整 llmMaxOutputTokens；分段執行複雜任務
ESP32-S3-CAM Watchdog	長阻塞的 SSL 傳輸或相機擷取可能觸發看門狗重啟	esp_task_wdt 明確餵食；長操作間穿插 yield

Token 用量與 System Prompt 成本

每一次 llmChatRequest() 與 llmSearchRequest() 呼叫，都會送出完整的 systemContentTools 提示詞——包含角色定義、所有已確認的裝置對應、硬體安全規則、技能工作流程腳本與完整的工具路由 schema——再加上開機以來累積的完整對話歷史。此成本結構對 Gemini／OpenAI／Grok 三種 llm_type 一致適用（實際計費費率依供應商而異）。在記憶體充裕但 API 預算有限的裝置上，這會帶來累積性的成本後果：

- 單次呼叫開銷：systemContentTools 提示詞本身在計入使用者訊息或對話歷史之前，即可能超過數千 token；對簡單問候或事實性問題而言，這樣的開銷純屬浪費。
- 歷史持續成長：historicalMessages 採只增不減的方式累積，目前沒有滑動視窗或摘要機制，每次後續呼叫都會原封不動送出整段歷史。
- 自主工作流程下的成本放大：evaluateWorkflowContinuation() 在每次工具執行後會觸發額外的 LLM 呼叫；多步驟工作流程中，單一使用者請求可能導致四到六次 API 往返，單一訊息的 Token 帳單因此可能是樸素估算的五到十倍。

- 沒有依複雜度分級的 Prompt 路由：程式碼庫中存在三組編譯好的 system prompt（systemContent、systemContentNoTools、systemContentTools），但主要訊息處理函式目前一律選用 systemContentTools，無論使用者輸入是否與硬體相關；此優化在架構上相當直接，但目前尚未實作。
- 模型選擇仍為手動，而非自適應：雖然 llm_type／llm_model 現已可於執行期設定，韌體尚未依請求複雜度自動選擇較輕或較重的模型，亦不會依成本或延遲自動切換供應商。

i 部署前檢查清單

在成本敏感或高頻率的環境中部署 fuClaw，建議將本節與第 4.5 節（Agent 對 Agent 通訊）視為擴展規模前的檢查清單：架構本身穩健，只需將成本結構與實際使用模式相互配對。

附錄 快速參考

A. GPIO 腳位對照表

開發板	裝置	腳位	控制模式	有效範圍
AMB82-mini	綠色 LED	24	digitalwrite	0 / 1
AMB82-mini	藍色 LED	23	digitalwrite	0 / 1
AMB82-mini	DHT11	8	dht11 (專屬工具)	—
AMB82-mini	窗戶致動器 SG90	5	servo (專屬工具)	0–180°
HUB 8735 Ultra	綠色 LED	25	digitalwrite	0 / 1
HUB 8735 Ultra	藍色 LED	26	digitalwrite	0 / 1
HUB 8735 Ultra	補光 LED	13	analogwrite	0–255
HUB 8735 Ultra	功能按鈕	12	digitalread (active-low)	0=按下
HUB 8735 Ultra	DHT11	20	dht11 (專屬工具)	—
HUB 8735 Ultra	窗戶致動器 SG90	12	servo (專屬工具)	0–180°
外部模組	緊急按鈕	1	digitalread (active-high)	1=按下
外部模組	光線感測器	2	analogread	0–1023
外部模組	警示燈	11	analogwrite	0–255 (預設 255)

i ESP32-S3-WROOM-CAM 硬體資源

GPIO SET : 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,38,39,40,41,42,43,44,45,46,47,48

ADC : 1–20 ; PWM : 0–21、38–48 (實際可用腳位仍須依 device.md 明確確認清單為準) 。

外部模組 (緊急按鈕、光線感測器、警示燈、SG90、DHT11) 之腳位定義與功能規則與 AmebaPro2 系列完全共用同一份 device.md 語意。

OLED 版變體 (檔名含 _OLED) 另配備 SSD1306 128×64 OLED，I2C 軟體匯流排：SDA=GPIO1、SCL=GPIO2，透過 /oled 工具控制 (見 4.3 節)；此硬體為 ESP32-S3-CAM 專屬，AmebaPro2 系列不提供。

B. 擴充感測器檢查清單

- device.md：加入感測器描述、腳位映射、物理規則 (範圍、時序)
- toolsDefinition：加入工具 JSON 請求格式與成功/錯誤回應格式
- 韌體：撰寫 tool_xxx() 驅動函式，包含錯誤處理

- 韌體：在 `executeTool()` 加入 `else if` 分支
- 韌體：在 `setup()` 加入初始化程式碼
- 燒錄並測試：確認 **Serial Monitor** 無錯誤，工具回應格式正確
- 如需跨平台部署，確認 **AmebaPro2／ESP32-S3-CAM** 對應函式庫（**AmebaServo/ESP32Servo**、**AmebaFatFS/SD_MMC** 等）皆已正確替換

C. 常見錯誤代碼

錯誤原因	錯誤碼	處置方式
digitalwrite 數值不是 0 或 1	invalid_digital_value	確認 LLM 輸出的 value 欄位
servo 角度超出 0–180	invalid_servo_angle	在 soul.md 強調角度範圍限制
servo 腳位未定義	undefined_servo_pin: <pin>	在 device.md 加入對應腳位定義
DHT11 讀值失敗	dht11_read_failed	確認接線；兩次讀取間隔 ≥1 秒
JSON 解析失敗	(Serial 輸出) JSON parse failed	降低 llmTemperature；回覆 Continue 重試
記憶體不足	malloc failed	立即執行 /reset；考慮 /reboot
LLM API 逾時	Connection failed	確認 WiFi 品質；確認 API Key 配額
MQTT/Telegram 傳送失敗	(工具回傳 error)	確認 env.json 憑證；確認 Broker/Bot Token 有效

D. 韌體變體與檔名對照表

fuClaw 以「2 平台 × 2 傳輸」提供 4 種可直接燒錄的韌體變體，四者共用完全相同的 LLM 推理引擎（Gemini／OpenAI／Grok 可選）、工具派發器與持久化記憶設計：

	Telegram Bot	MQTT
Realtek AmebaPro2 (RTL8735B)	AmebaPro2_fuClaw_TelegramBot_SD_SG90_DHT11.ino	AmebaPro2_fuClaw_MQTT_SD_SG90_DHT11.ino
ESP32-S3-WROOM-CAM	ESP32S3_CAM_fuClaw_TelegramBot_SD_SG90_DHT11_OLED.ino	ESP32S3_CAM_fuClaw_MQTT_SD_SG90_DHT11_OLED.ino

若要套用至新的部署情境，開發者通常只需修改 `soul.md` 與 `device.md`；核心架構無需重新設計。跨硬體移植所需修改的範圍，請參見 1.5 節五個硬體抽象點對照表。

檔名中的 `_OLED` 後綴標示該韌體變體另配備 **SSD1306 OLED** 顯示模組與 `/oled` 工具（見 4.3 節）；此模組目前僅 **ESP32-S3-CAM** 版本提供，**AmebaPro2** 系列（**AMB82-mini／HUB 8735 Ultra**）尚無對應硬體，故其檔名不含此後綴。`llm_type` 供應商選擇（見 2.7、3.1 節）與韌體變體正交，四種變體皆可透過 `env.json` 自由切換 **Gemini／OpenAI／Grok**，無需更換 `.ino` 檔案。

